Pontificia Universidad JAVERIANA Colombia

Aplicaciones con LLMs

Semana 1 · LangChain paso a paso
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3

RAG y evaluación
Semanas 4–6

Model Context Protocol
Semana 7

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 1 · LangChain paso a paso

- Por qué construir con LLMs y no solo usarlos
- Tokens, contexto e inferencia
- Mensajes, plantillas y salidas estructuradas
- `invoke`, `batch` y `stream`
- Cadenas con `@chain` y `|`
- Laboratorio: L01 y L02

De usar IA a construir con IA

Motivación

El cambio que buscamos

1. Usar: pedimos una respuesta y la revisamos a mano.
2. Construir: diseñamos el flujo completo alrededor del modelo.
3. Decidimos qué información recibe y qué acciones puede pedir.
4. Decidimos qué comprueba nuestro código y cuándo entra una persona.

El caso del curso: ACME

Una fintech ficticia

Datos sintéticos

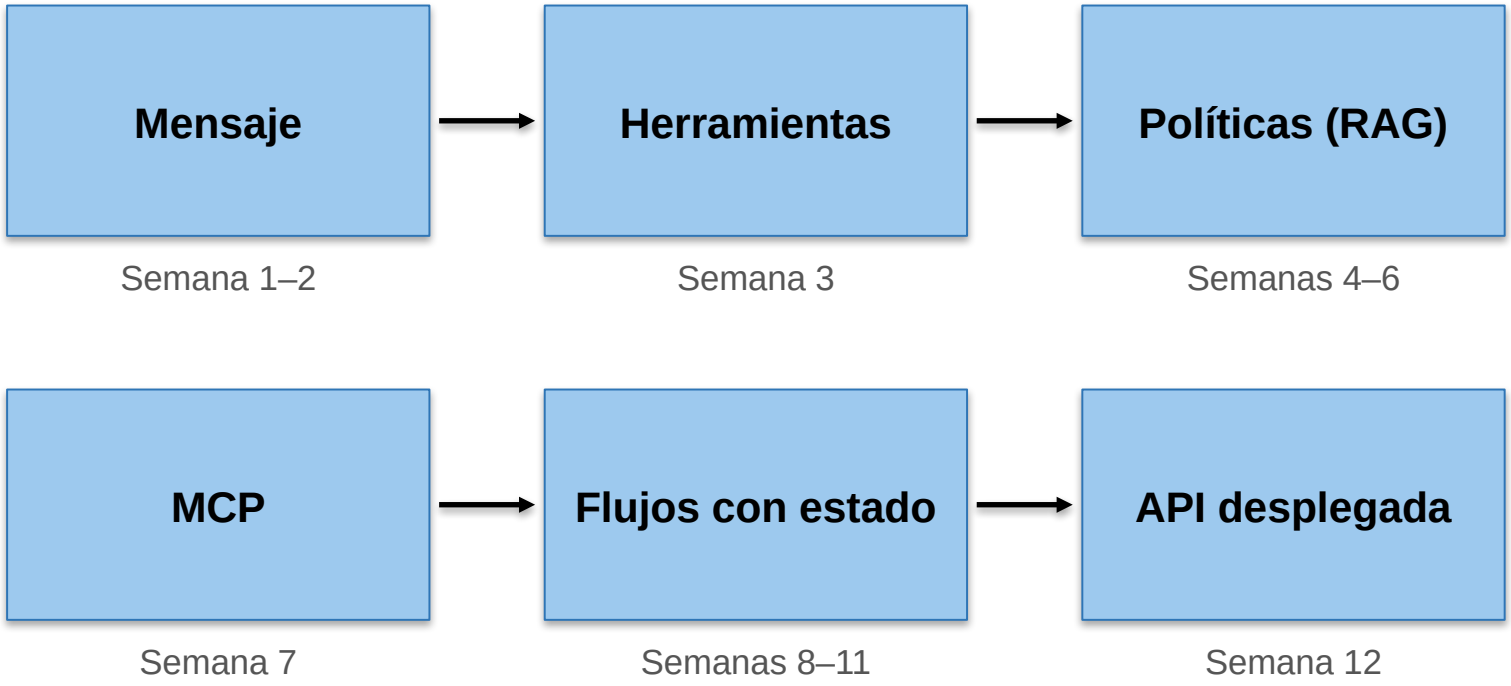
ACME ofrece créditos, tarjetas y desembolsos.

Su equipo de soporte recibe cientos de mensajes al día: cargos duplicados, mora, desembolsos, tarifas.

Durante 12 semanas construiremos su asistente.

Lo que construiremos en el curso

Del mensaje al agente desplegado



Cada semana añade una pieza al mismo asistente de soporte de ACME.

Workflows y agentes

Dos formas de construir

Concepto	Qué lo define
Workflow	El código fija los pasos; el modelo puede decidir dentro de cada uno
Agente	El modelo elige sus siguientes acciones y herramientas
Bloque básico	Un LLM con recuperación, herramientas o memoria

Los patrones que veremos

Estructuras fijadas en código

Patrón	Cuándo usarlo	Semana
Ciclo de agente	El modelo decide qué herramienta usar	3 y 7
Prompt chaining	La tarea se divide en pasos conocidos	8
Routing	Categorías con tratamientos distintos	8
Parallelization	Subtareas independientes que se agregan	8
Human-in-the-loop	Una acción necesita aprobación	9
Orchestrator-workers	Las subtareas dependen de la entrada	10
Evaluator-optimizer	Hay criterios explícitos para mejorar	11

Pontificia Universidad JAVERIANA Colombia

El modelo

Qué pasa dentro de una llamada

Texto y tokens

Las unidades que procesa el modelo

Texto	Representación ilustrativa
cargo duplicado	[cargo] [dup] [licado]
185.000 COP	[185] [.000] [COP]
Los cortes reales	dependen del tokenizador de cada modelo

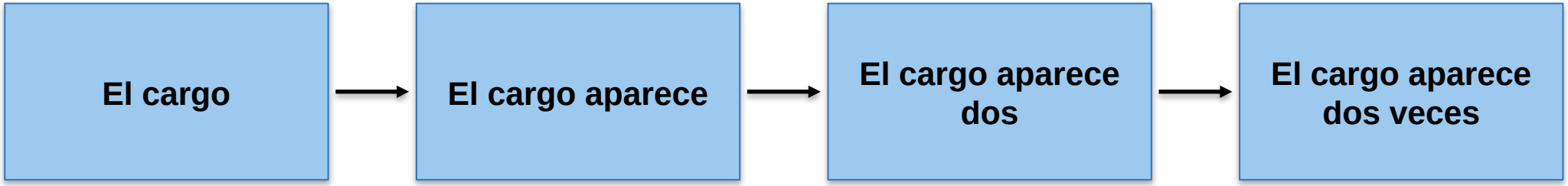
La ventana de contexto

Todo lo que el modelo lee en una llamada

Contenido	Ejemplo en ACME
Instrucciones de sistema	Responde solo con el ticket; si falta el dato, dilo
Conversación	Los mensajes anteriores del cliente y del asistente
Datos	El ticket T-2048, una política, un registro de la cartera
Resultados de herramientas	El monto vencido que devolvió una consulta
La respuesta	También ocupa tokens: el límite es `max_tokens`

La generación paso a paso

Predecir el siguiente token



En cada paso el modelo elige un token según una distribución de probabilidad.

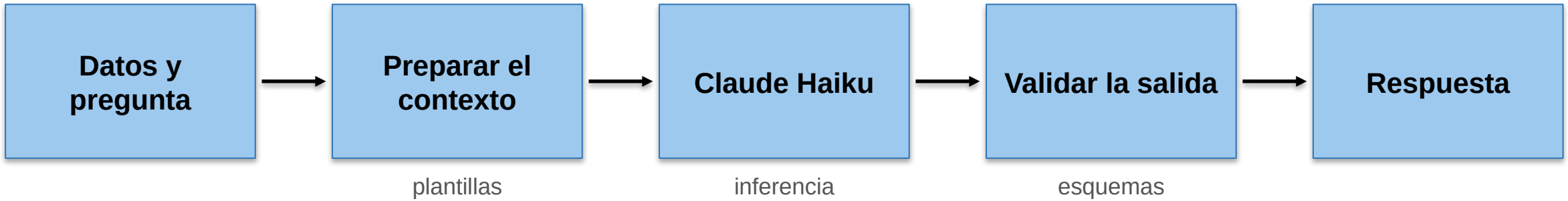
Entrenamiento, inferencia y contexto

Tres conceptos que conviene separar

Proceso	Qué cambia
Entrenamiento	Los parámetros del modelo
Inferencia	La salida generada en cada llamada
Contexto de la llamada	La información que suministramos: instrucciones, datos, historial

Una aplicación con un modelo

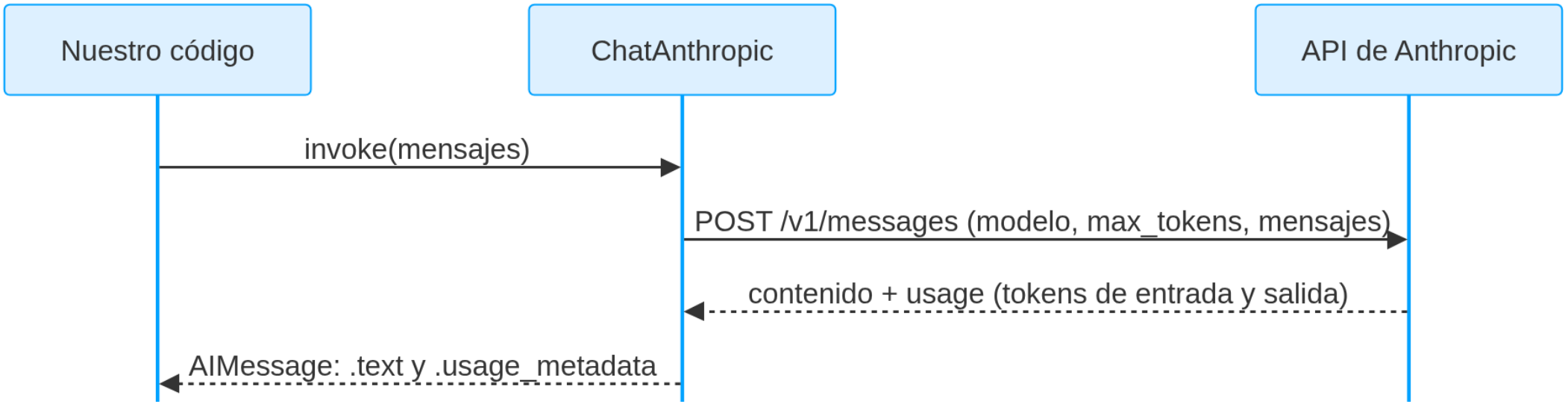
El modelo es un componente



La aplicación prepara el contexto antes de llamar y comprueba la salida después.

Qué pasa en una llamada

Diagrama de secuencia



Una llamada. LangChain traduce los mensajes al formato de la API y devuelve un `AIMessage` con el texto y los tokens usados.

Pontificia Universidad JAVERIANA Colombia

LangChain

Mensajes, plantillas y salidas

La primera llamada

L01 · secciones 0 y 1

LangChain y Claude

La clave ANTHROPIC_API_KEY vive en .env, nunca en el notebook.

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(
    model="claude-haiku-5-5",
    max_tokens=400,
    thinking={"type": "disabled"},
)
ticket_text = ("Ticket T-2048: el cliente C-1042 de ACME "
               "reporta dos cargos iguales de 185.000 COP...")
response = model.invoke(
    f"Resume en una frase este mensaje: {ticket_text}")
print(response.text)
```

Los parámetros del modelo

Lo que configura ChatAnthropic

Parámetro	Valor en el curso	Para qué
`model`	`claude-haiku-5-5`	El modelo más pequeño y barato de la familia
`max_tokens`	400–600	Tope de la respuesta; se paga lo generado
`thinking`	`{"type": "disabled"}`	Sin razonamiento extendido: no gasta el tope en pensar
`timeout`, `max_retries`	60 s, 2	Una llamada colgada no bloquea el programa
`temperature`	No se usa	Haiku 5.5 no la acepta

Mensajes con roles

L01 · secciones 2 y 3

Mensaje	Papel	Ejemplo
`SystemMessage`	Instrucciones generales	Responde con una frase que empiece por 'Problema:'
`HumanMessage`	La solicitud del usuario	El ticket y la pregunta
`AIMessage`	La salida del modelo	`response.text` y metadatos de tokens

Plantillas de mensajes

L01 · secciones 4 a 7

Separar instrucciones y datos

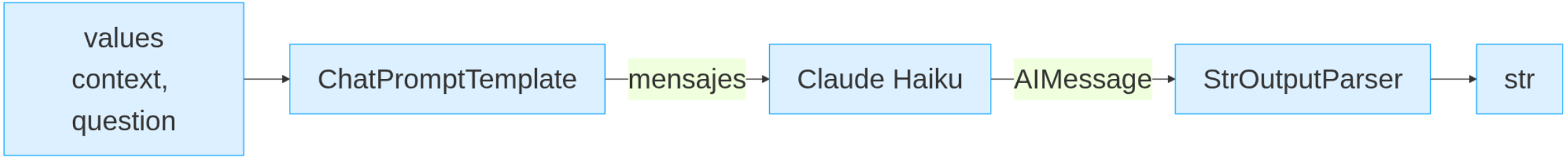
Preparar la plantilla
no consume tokens.

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate.from_messages([
    ("system", "Responde brevemente usando solo el "
               "contexto. Si falta el dato, responde "
               "'No se indica en el ticket'."),
    ("human", "Contexto: {context}"),
    ("human", "Pregunta: {question}"),
])
values = {"context": ticket_text,
          "question": "¿De cuánto es cada cargo?"}
chat_prompt = chat_template.invoke(values)  # no llama al modelo
response = model.invoke(chat_prompt)
```

Del dato a la respuesta

Plantilla, modelo y parser



Una cadena. `format_messages` llena la plantilla sin crear una traza; `invoke` también la llena, pero cuenta como una.

Una salida estructurada

L01 · sección 8

Pydantic y LangChain

Las descripciones
de los campos también
son instrucciones.

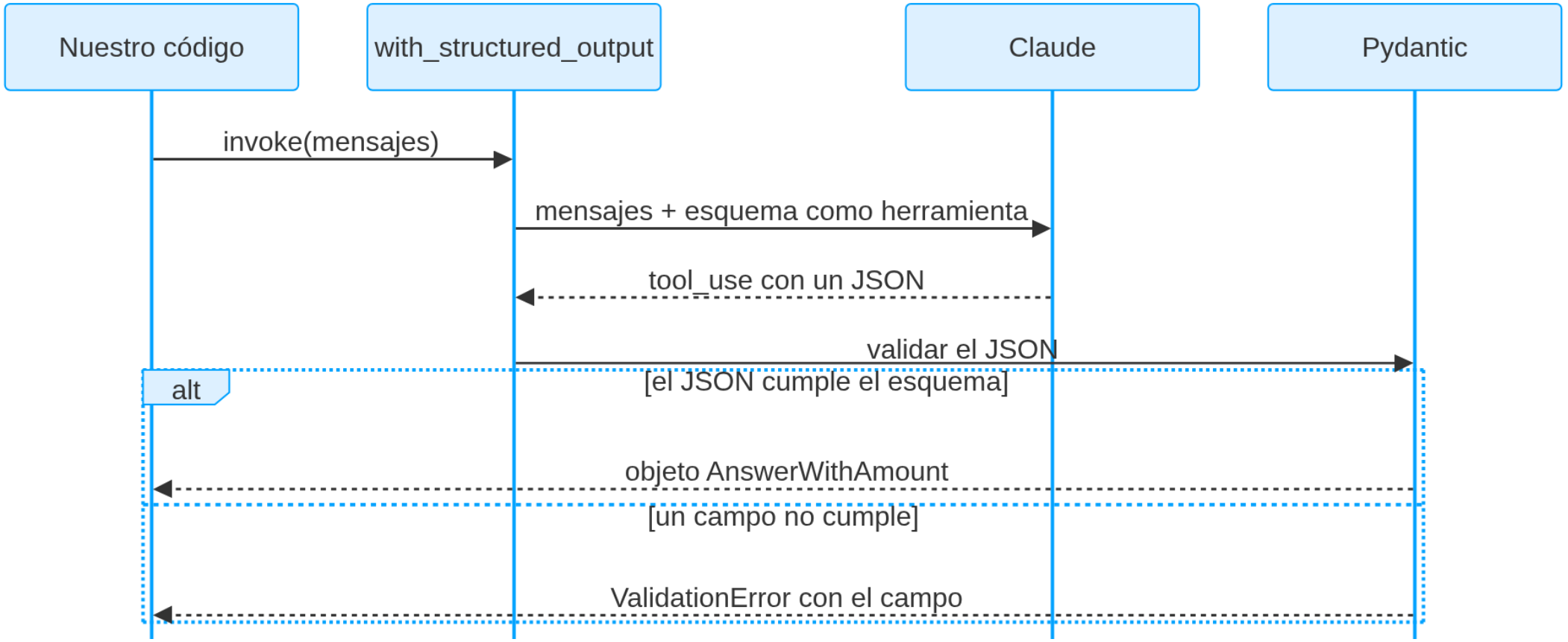
```
from pydantic import BaseModel, Field

class AnswerWithAmount(BaseModel):
    """Respuesta breve sobre el ticket."""
    answer: str = Field(description="Respuesta a la pregunta.")
    justification: str = Field(
        description="Dato del ticket que la sustenta.")
    amount: int = Field(
        description="Monto de cada cargo en pesos.")

amount_model = model.with_structured_output(AnswerWithAmount)
amount_model.invoke(chat_prompt)
# answer='Cada cargo es de 185.000 COP.' ... amount=185000
```

Cómo se obtiene una salida estructurada

Diagrama de secuencia



Esquema y validación. El esquema viaja como una herramienta; Pydantic valida el JSON que vuelve y nombra el campo que falla.

Categorías cerradas y datos ausentes

L02 · secciones 2 y 3

Diseñar el esquema

`Literal` evita categorías inventadas. `None` evita fechas inventadas.

```
from typing import Literal

class TicketTriage(BaseModel):
    """Clasificación de un mensaje de soporte de ACME."""
    customer: str = Field(description="Código C-1234.")
    category: Literal["cargo_duplicado", "bloqueo_tarjeta",
                      "desembolso", "mora", "otro"]
    urgency: Literal["alta", "media", "baja"]
    refund_requested_cop: int | None = Field(
        description="None si no pide devolución.")
    charge_date: str | None = Field(
        description="AAAA-MM-DD; None si no hay fecha exacta.")
```

Validar reglas de negocio

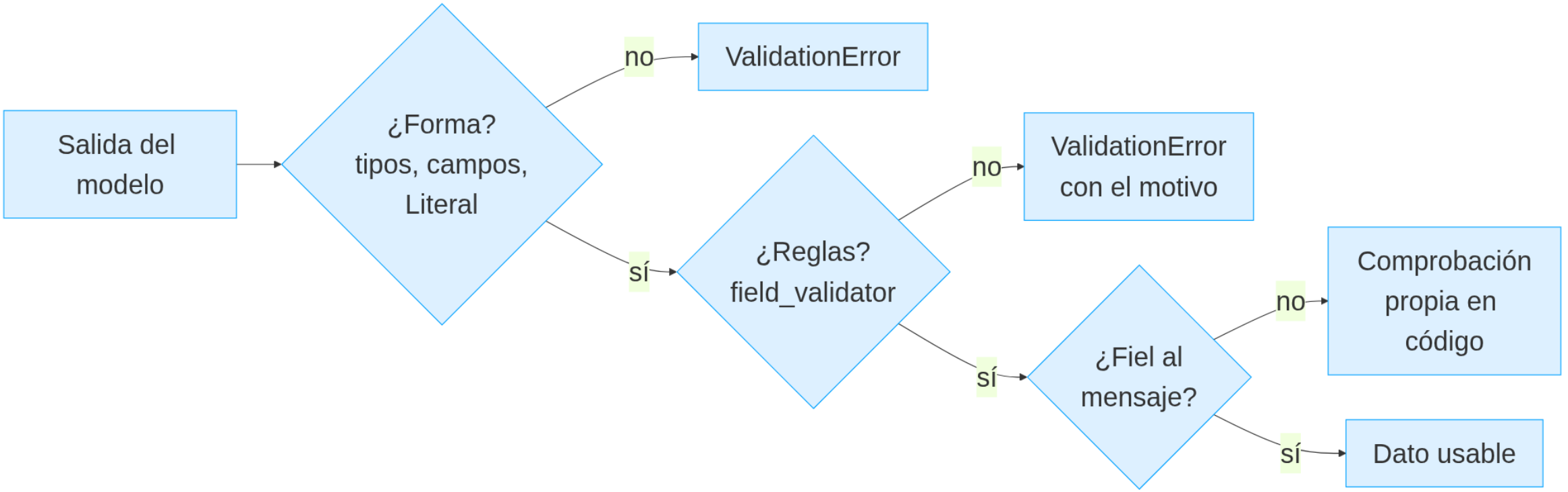
L02 · secciones 4 a 6

Forma y contenido

1. Listas de objetos: un `Charge` por cada cargo que menciona el cliente.
2. `field_validator`: el código del cliente debe tener el formato C-1234.
3. Si la validación falla, `ValidationError` dice qué campo y por qué.
4. `include_raw=True` deja ver la respuesta cruda del modelo.
5. Un esquema válido no garantiza que el dato sea **fiel** al mensaje.

Tres comprobaciones distintas

Forma, reglas y fidelidad



Validar por capas. Pydantic cubre la forma y las reglas que se escriben en código; la fidelidad al mensaje necesita una comprobación propia.

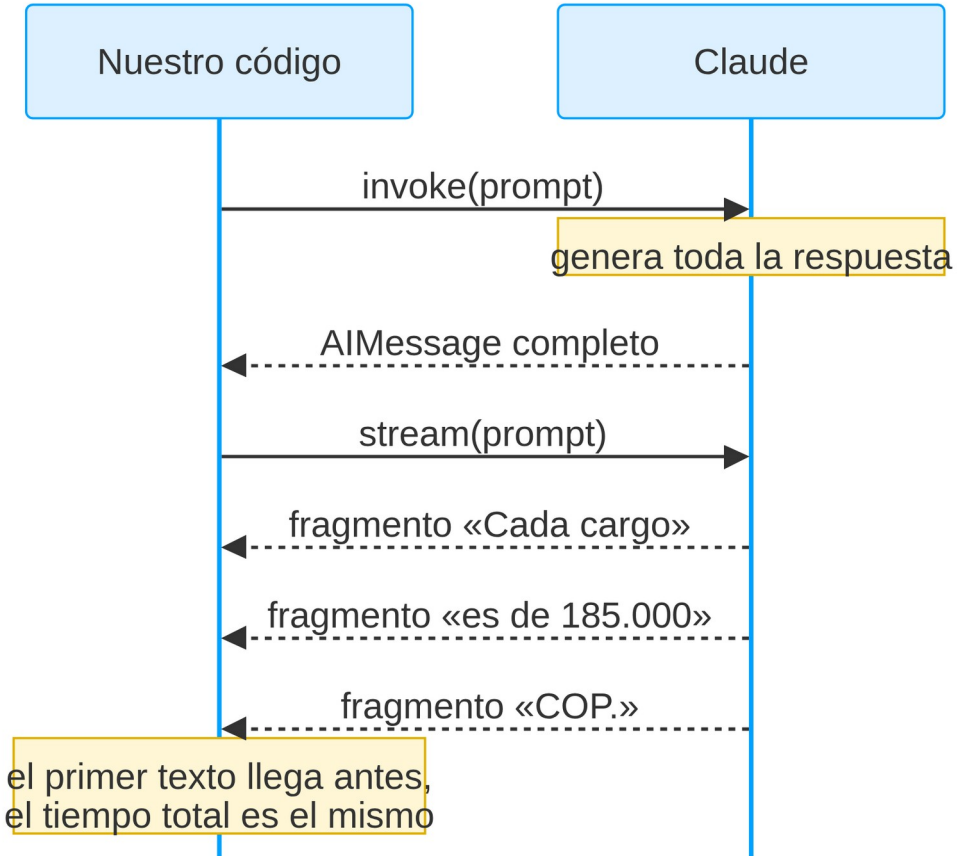
invoke, batch y stream

L01 · sección 10

Método	Qué hace
`invoke`	Una entrada y una respuesta completa
`batch`	Varias entradas en paralelo, en el mismo orden
`stream`	Una entrada; la respuesta llega por fragmentos
`batch` ≠ Batch API	Son llamadas normales, no diferidas ni con descuento

invoke frente a stream

Diagrama de secuencia



Mostrar antes. `stream` entrega fragmentos mientras se generan: la persona lee antes, pero la respuesta no termina antes.

Cadenas con @chain y |

L01 · secciones 11 a 14

Componer pasos

Una cadena crece añadiendo pasos.

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import chain

@chain
def support_assistant_text(inputs):
    prompt = chat_template.invoke(inputs)
    response = model.invoke(prompt)
    return StrOutputParser().invoke(response)

# La misma cadena, de forma declarativa
assistant_text = chat_template | model | StrOutputParser()
assistant_text.invoke(values)  # 'Cada cargo es de 185.000 COP.'
for chunk in assistant_text.stream(values):
    print(chunk, end="")
```

La interfaz Runnable

Lo que comparten plantillas, modelos, parsers y cadenas

Método	Qué hace
`invoke` / `ainvoke`	Una entrada, una salida; la segunda espera sin bloquear
`batch`	Varias entradas en paralelo, en el mismo orden
`stream` / `astream`	La salida por fragmentos
`|`	Conecta componentes: la salida de uno es la entrada del siguiente
`@chain`	Convierte una función en un componente con los mismos métodos

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L01-IntroToLLMs · L02-StructuredOutput

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. L01: llamar a Claude con texto, mensajes y plantillas (18 llamadas).
2. L01: salida estructurada, `batch`, `stream` y cadenas.
3. L02: diseñar el esquema de un ticket con categorías y datos ausentes.
4. L02: validar reglas de negocio y leer la respuesta cruda.
5. Práctica: cambiar el ticket por otro caso de ACME.

¿Cómo sabremos que funcionó?

Criterios de logro

La prueba del ejercicio

1. La respuesta usa solo los datos del ticket.
2. Un dato ausente produce 'No se indica en el ticket', no un valor inventado.
3. La salida estructurada respeta el esquema y coincide con el mensaje.
4. Podemos explicar qué cambia entre `invoke`, `batch` y `stream`.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Una plantilla prepara el prompt; no llama al modelo.
2. La estructura facilita usar la respuesta, pero no garantiza su fidelidad.
3. `stream` muestra antes; no responde más rápido.
4. Una cadena crece añadiendo pasos.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 1	Modelos, plantillas, salidas y cadenas
AI Engineering, cap. 2	Cómo funcionan los modelos fundacionales
Anthropic, Building effective agents	Workflows, agentes y cuándo usar cada uno
Docs de LangChain	Structured output y runnables

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Prompts dinámicos con **Jinja**, esquemas y trazas en **LangSmith**.
Notebook: `L03-DynamicPrompting.ipynb`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso